

API Schema

Overdrive

Aventix
Epitech Paris
February 9, 2026

Summary

OverDrive – API Schema

1- General Principles: REST vs WebSocket

- 1- General principles: REST vs WebSocket
- 2- WebSocket: Live streaming
- 3- Diagram: Overall architecture (REST + WebSocket)

2- API REST : endpoints probables

- 1- Authentication
- 2- Providers and course catalog
- 3- Presets and layouts
- 4- Preset widgets
- 5- Highlights (optional)
- 6- Diagram: Data model
- 7- Diagram: Authentication sequence

3- WebSocket: real-time streaming

- 1- Connection
- 2- Types of messages sent
- 3- Load management and optimization
- 4- Diagram: WebSocket connection + session subscription
- 5- Diagram: WebSocket message format
- 6- Diagram: Load optimization (differentiated broadcasting)

4- Recommended technical principles

- 1- Versioning
- 2- Documentation (OpenAPI / Swagger)
- 3- Validation and JSON schemas
- 4- Rate limiting (REST)
- 5- Standardized error management
- 6- Access control

5-Conclusion

Objective

The OverDrive API is designed to meet two major and complementary needs :

- **Manage the product and persistent data** (accounts, presets, race catalog, user configuration).
- **Broadcast live data in real time** (ranking, telemetry, racing events) with minimal latency.

To achieve these goals, OverDrive adopts a standard and robust architecture based on:

- a **REST API** for all standard and persistent operations,
- a **WebSocket** channel for real-time streaming.

This separation is essential in a motorsport application, because it avoids overloading the REST API with repeated requests (polling), while ensuring a smooth experience on all customers (VR, mobile, web).

1- General principles: REST vs WebSocket

1- REST API: product management and configuration

The REST API is used for all operations that do not require constant updating, including :

- Authentication,
- user profile retrieval,
- preset and layout management,
- course and session catalog,
- provider configuration,
- access to persistent data (highlights, replays, etc.).

REST is particularly well-suited because it is:

- Simple to version,
- easy to document,
- compatible with all clients,
- stable for CRUD (Create, Read, Update, Delete) operations.

2- WebSocket: live streaming

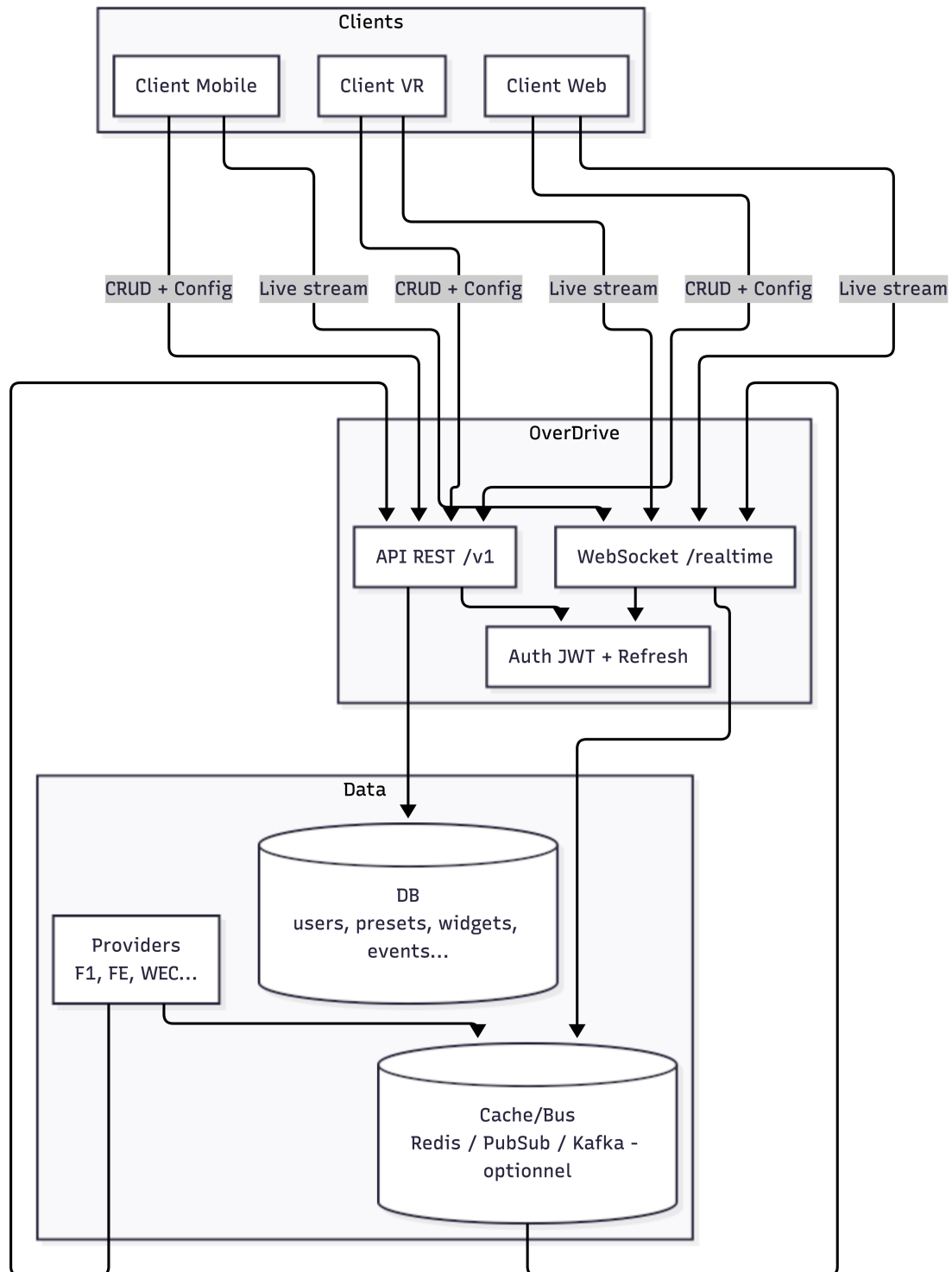
WebSocket is used for real-time streaming because it allows:

- a persistent connection,
- automatic sending of updates without client request,
- reduced latency,
- optimal operation for very frequent data.

This channel is used to broadcast:

- Session status (lap, flags, weather, track status),
- ranking updates,
- gaps,
- telemetry,
- highlight events.

3- Diagram: Overall architecture (REST + WebSocket)



2- API REST : points d'extrémité probables

The OverDrive REST API is organized around large functional blocks.

1- Authentication

Authentication covers essential user management operations :

- Account creation,
- login,
- logout,
- profile recovery.

Main endpoints :

- **POST /auth/register**
- **POST /auth/login**
- **POST /auth/logout**
- **GET /me**

Authentication can be based on :

- an access token (JWT type),
- a refresh token mechanism to maintain a user session without compromising security.

This system allows for a good balance between :

- Security (short expiration of access tokens),
- user convenience (refresh token).

2- Providers and race catalog

Since OverDrive is a multi-championship game, the customer must be able to access a structured catalogue.

Main endpoints :

- **GET /providers (liste des championnats disponibles)**
- **GET /events?provider=f1&year=2026**
- **GET /events/{eventId}**
- **GET /events/{eventId}/sessions**

These roads allow :

- to display an annual calendar,
- to browse past events,
- to prepare a session to follow live,
- to link sessions (practice/qualification/race) to an event.

3- Presets et layouts

Since personalization is a central element of OverDrive, preset management must be comprehensive and accessible from all clients.

Main endpoints :

- **GET /presets**
- **POST /presets**
- **GET /presets/{presetId}**
- **PUT /presets/{presetId}**
- **DELETE /presets/{presetId}**

These endpoints allow :

- to create multiple presets,
- to save and restore layouts,
- to define a default preset,
- to ensure consistency between VR, mobile and web.

4- Widgets d'un preset

Chaque preset est composé d'un ensemble de widgets configurables. Le client doit pouvoir gérer ces widgets de manière structurée.

Endpoints principaux :

- **GET /presets/{presetId}/widgets**
- **POST /presets/{presetId}/widgets**
- **PUT /presets/{presetId}/widgets/{widgetId}**
- **DELETE /presets/{presetId}/widgets/{widgetId}**

These endpoints cover :

- Adding widgets,
- deleting widgets,
- updating settings,
- saving VR positions and transformations.

This approach guarantees complete customization without depending on a specific client.

5- Highlights (optional)

If the backend stores important events in the database, a dedicated endpoint allows access to them.

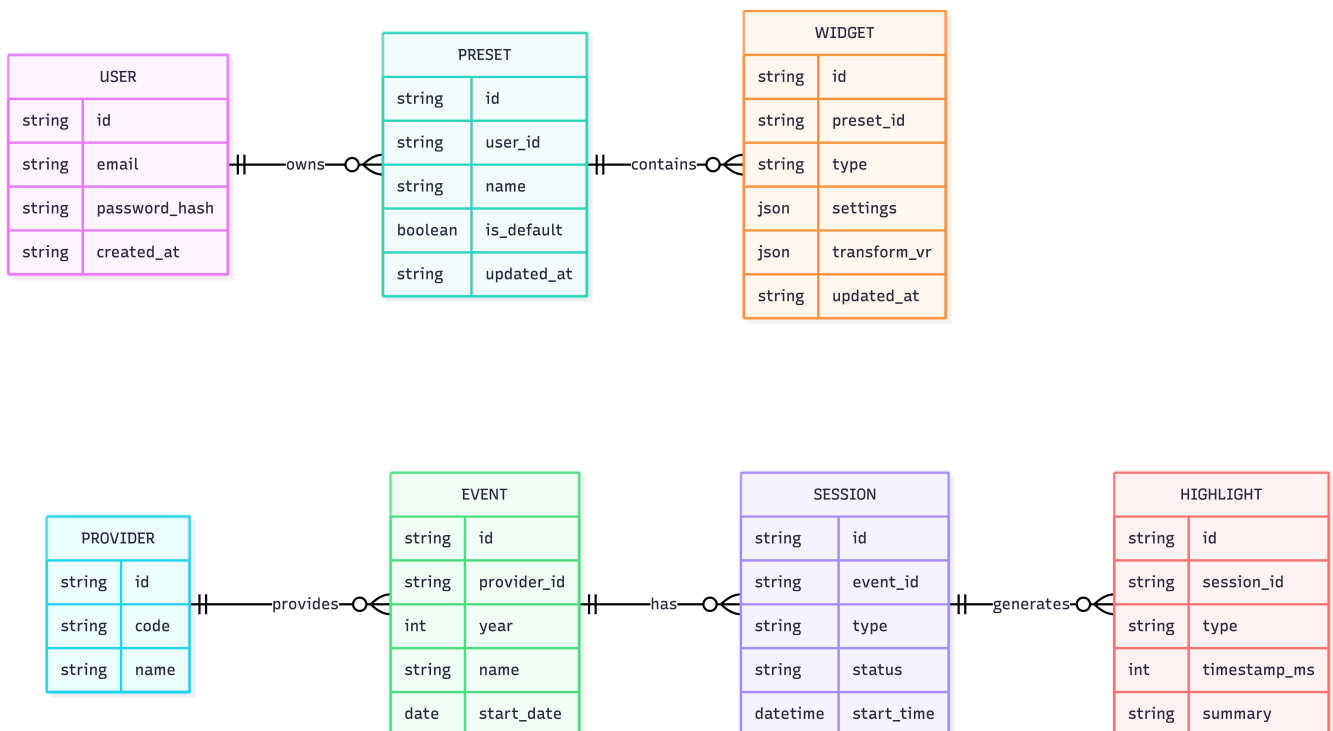
Main endpoint:

- **GET /sessions/{sessionId}/highlights**

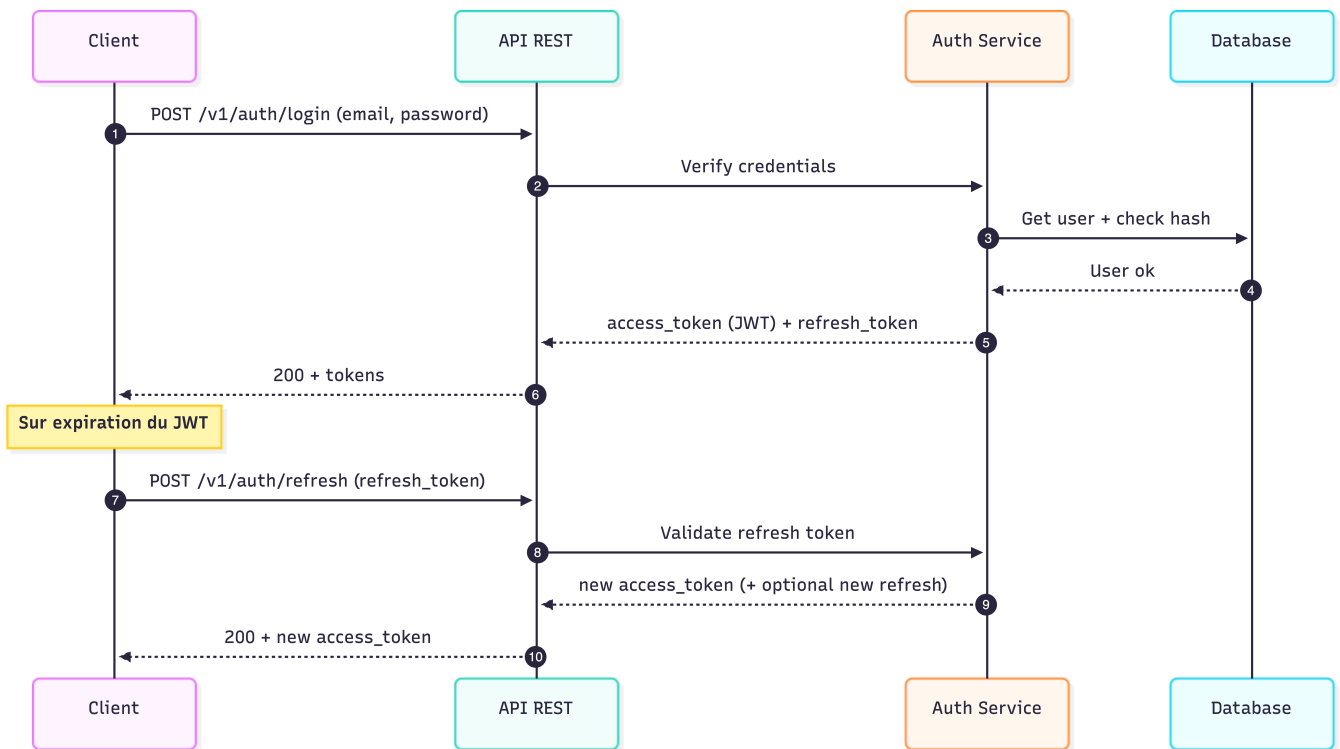
This route allows you to display :

- a timeline of key moments,
- a structured summary of the session,
- a platform for replays or analysis.

6- Diagram: Data Model



7- Diagram: Authentication Sequence



3- WebSocket: real-time streaming

WebSocket is the primary channel for live streaming. It replaces REST polling, which is too expensive and too slow for frequent updates.

1- Login

The WebSocket connection is made over a dedicated route :

- **WS /realtime?sessionId=...**

During connection, the client provides :

- A session identifier (sessionId)
- An access token (to authenticate the user)

This mechanism allows :

- to secure access to the live stream,
- to correctly associate the stream with a specific session,
- to prevent a client from listening to unauthorized sessions.

2- Types of messages sent

The backend sends structured, typed messages so that they can be easily interpreted by clients.

session_state

This message represents the overall state of the session.

Examples of information :

- current lap,
- current flag,
- weather,
- track status,
- overall status (live, finished).

standings_update

This message corresponds to ranking updates.

It may contain :

- positions,
- gaps,
- changes in position,
- strategy information (if available).

telemetry_update

This message corresponds to a pilot's telemetry.

Typical fields :

- driver_id
- timestamp_ms
- speed
- gear
- throttle
- brake
- (and any available metrics)

highlight_event

This message corresponds to an important event detected or received from the provider.

Typical fields:

- type (overtake, crash, pit, flag)
- timestamp_ms
- summary

3- Workload management and optimization

A major constraint of real-time systems is server load. To avoid overload, OverDrive can apply a streaming logic.

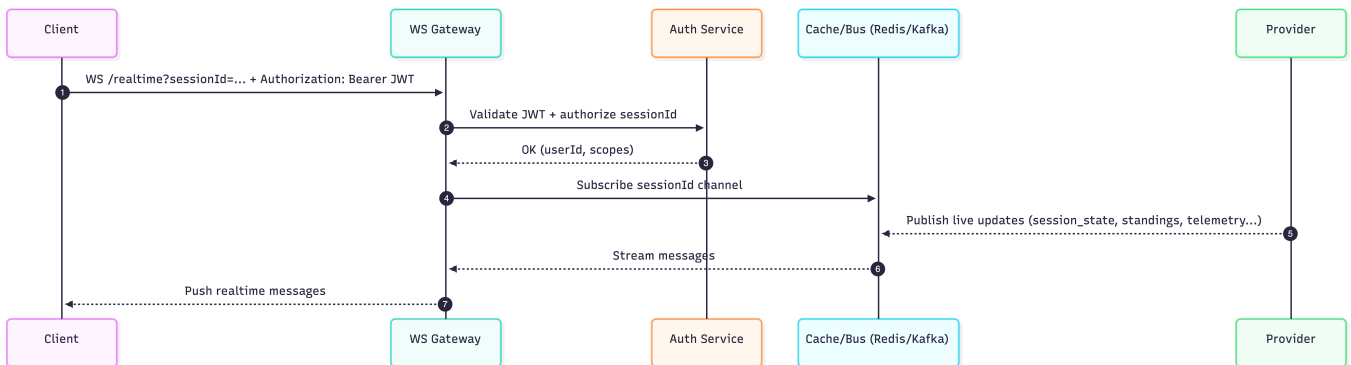
Differentiated :

- High frequency for the "focus" pilot (e.g., following the favorite pilot),
- reduced frequency for other pilots,
- limitation of certain messages depending on the context (e.g., VR vs. mobile).

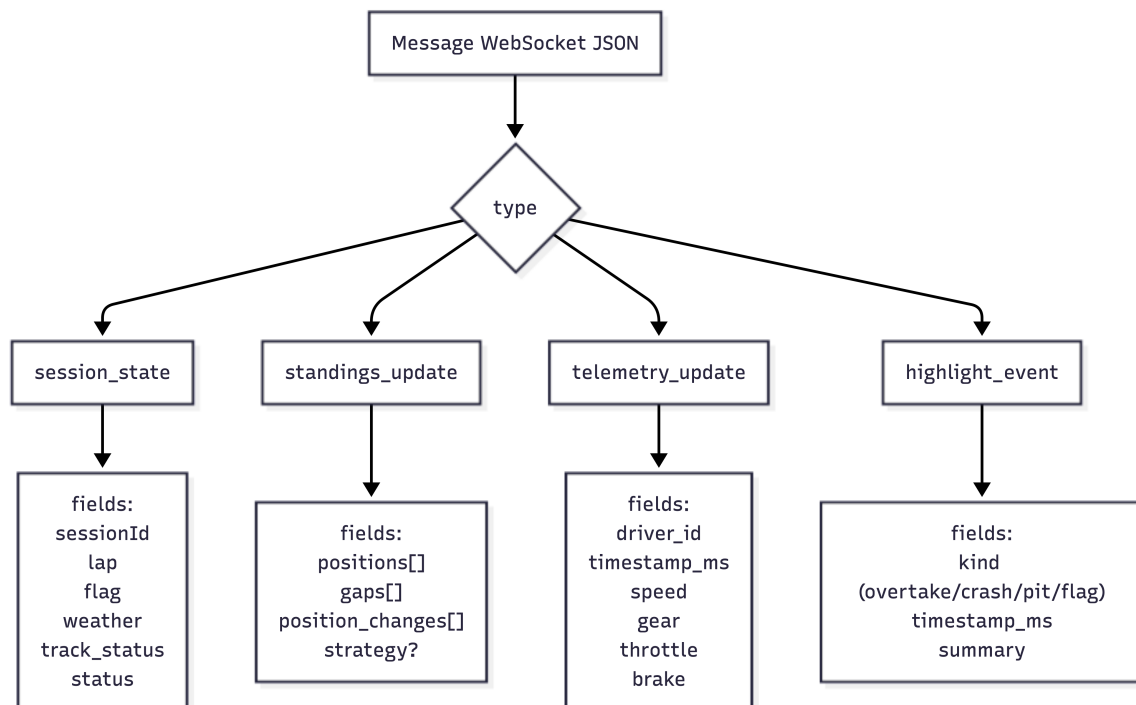
This helps to maintain :

- A smooth experience,
- controlled bandwidth,
- a reasonable server cost.

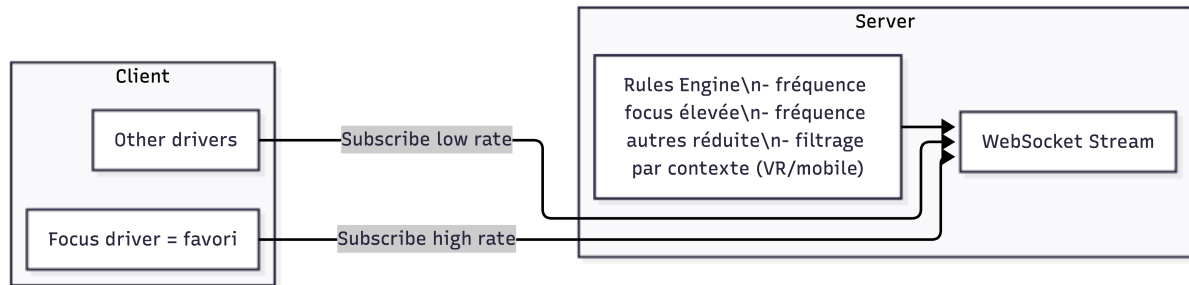
4- Diagram: WebSocket connection + session subscription



5- Diagram: WebSocket message format



6- Diagram: Load optimization (differentiated diffusion)



4- Recommended technical principles

To make the API stable, maintainable and scalable, several best practices are recommended.

1- Versioning

All REST routes must be versioned, for example :

- **/v1/auth/...**
- **/v1/events/...**
- **/v1/presets/...**

Versioning allows:

- to evolve without breaking customers,
- to manage multiple versions if necessary.

2- Documentation (OpenAPI / Swagger)

The REST API must be documented using a standard such as OpenAPI, in order to :

- Facilitate multi-client development,
- enable rapid understanding of endpoints,
- guarantee consistency between backend and clients.

3- Validation and JSON schemas

The backend must validate systematically :

- User input (REST),
- WebSocket payloads.

This reduces the risk of :

- bugs,
- inconsistencies,
- vulnerabilities (injection, crash, malformed payload).

4- Rate limiting (REST)

Even though WebSocket covers live traffic, REST must be protected against abuse :

- Rate limiting per IP address,
- rate limiting per user,
- specific limitations on **/auth/login**.

5- Standardized error handling

Errors must follow a consistent format, including :

- Clear HTTP code,
- a readable message,
- an error identifier if necessary,
- a structured explanation.

This simplifies debugging and improves product quality.

6- Contrôle d'accès

The API must guarantee :

- that a user can only access their presets,
- that certain routes are protected,
- that the WebSocket sessionId is authorized,
- that permissions are consistent (RBAC possible in the long term).

Conclusion

The proposed API scheme for OverDrive is based on a clear and standard architecture:

- **REST** for all persistent data and product management (auth, presets, catalogue, configuration),
- **WebSocket** to broadcast live data with minimal latency (rankings, telemetry, events).

This approach allows :

- Native compatibility with VR, mobile and web clients,
- a smooth and premium live experience,
- a scalable architecture (addition of providers, new widgets, new features),
- simplified maintenance thanks to best practices (versioning, OpenAPI, validation, security).

This framework provides a solid foundation for the development of OverDrive and its future evolution.